

Sparse Execution and Routing-Guided Pruning for Ultra-Lightweight Medical Image Segmentation

Research Proposal (Path A) : Methodological Enhancement

M. Mohaiminul Islam

This proposal studies one practical limitation of MK-UNet: its multi-kernel block always computes the same three branches for every input. The project tests whether input-dependent routing can maintain segmentation quality while reducing unnecessary computation, and whether these savings lead to lower measured inference latency.

1 Motivation

Portable medical devices often have limited computing resources and may not have a dedicated GPU. Therefore, small segmentation models are important when memory, computation, and latency are limited. MK-UNet was designed for this setting. Its standard model is reported with 0.316 M parameters and 0.314 G FLOPs, while MK-UNet-T has about 0.027 M parameters [1]. The paper also reports competitive segmentation results across six binary medical-image benchmarks. However, MK-UNet always computes all three depth-wise kernel branches $\{1, 3, 5\}$ and combines them using the fixed sum

$$F = F_1 + F_3 + F_5.$$

The BUSI ablation shows that kernel choice matters: DICE rises from 70.83 with only a 1×1 kernel to 78.04 with $\{1, 3, 5\}$ [1]. However, the same three branches are computed for every input and at every stage.

Different images may not need the same receptive fields. A large polyp, a small lesion, and an uncertain boundary may need different amounts of local and wider context. The main motivation of this proposal is therefore simple: if the useful kernel branch depends on the input, computing all branches every time may waste computation. This study asks whether MK-UNet can make this choice dynamically and whether skipping some branches gives a real latency benefit, not only a lower FLOP count.

Research question and objectives: Can MK-UNet select and run kernel branches based on the input while keeping good segmentation quality and its lightweight design? The study has three main objectives: (1) test input-dependent branch weighting, (2) test sparse top-1 routing that skips unused branches during inference, and (3) use routing statistics to find branches that can be removed from a smaller static model. Predictive entropy is also tested as an additional routing signal, not as the main novelty.

2 Related Work

The main problem is that MKDC has several useful receptive fields, but it always runs the same three branches. A useful extension must answer two questions: can the model choose the kernel branches based on the input, and can this choice reduce real inference cost after including the router cost?

Recent medical segmentation studies show that dynamic convolution is an active research direction. LightAWNet uses dynamic convolution in a lightweight segmentation network and reports parameter count, GFLOPs, and execution time [2]. AD-LA Former uses auto-dynamic convolution with location attention to adapt features to different inputs [3]. SEDyConv introduces input-dependent multi-dimensional dynamic convolution and dynamic switches for 3D multi-organ CT segmentation [4]. More recently, SGDC uses structure-guided dynamic kernels and gating to preserve fine boundary information [5]. These studies support the use of adaptive convolution for medical segmentation, but they do not report the same experiment studied here: explicitly skipping MK-UNet’s parallel depth-wise branches using top-1 routing in the ultra-small MK-UNet-T model.

The novelty is therefore not dynamic convolution itself. The open question is whether conditional multi-kernel execution is still useful when the full segmentation model has only about 0.027 M parameters [1]. At this small scale, the cost of the router and branch selection may be large compared with the backbone. This proposal focuses on actually skipping branches, using routing results for static pruning, and measuring batch-1 latency instead of assuming that lower FLOPs will give the same reduction in runtime.

Recent work	Main adaptive idea	Efficiency evidence	Sparse branch execution
LightAWNet [2]	Dynamic convolution	Params, GFLOPs, time	Not reported
AD-LA Former [3]	Auto-dynamic convolution	Internal scaling or redundancy reduction	Not reported
SEDyConv [4]	Dynamic convolution + switches	Small parameter growth	Not reported
SGDC [5]	Structure-guided dynamic kernels	Accuracy / boundary focus	Not reported
MK-UNet-T [1]	Fixed {1, 3, 5} MKDC	0.027M parameters, throughput	No
This proposal	Input conditioned top-1 routing	Executed FLOPs + batch1 latency	Yes

The preliminary measurements already show why runtime must be measured directly: in one sparse top-1 run, reported FLOPs fell by about 54%, while measured batch-1 CPU latency fell by about 13.3%.

3 Proposed Methodology

The proposed method changes MKDC step by step, so each experiment answers one clear question. The study first tests input-dependent branch weighting, then adds a difficulty signal, then moves to actual branch skipping, and finally uses the routing results to design a smaller static model. Figure 1 shows this process.

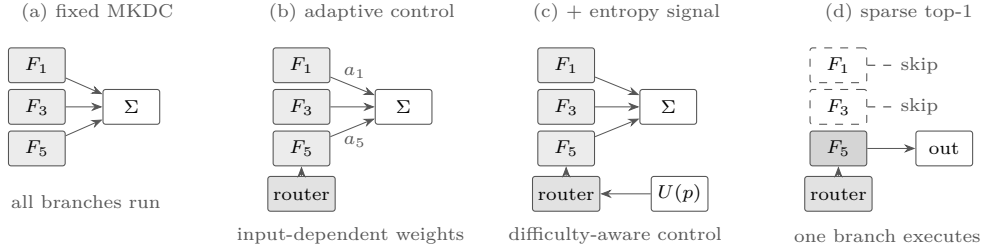


Figure 1: Adaptive and entropy-conditioned controls compute all three branches, while sparse top-1 executes only the selected branch.

A. Feature-adaptive soft routing: A small router uses global average pooling and two 1×1 layers to predict three branch weights. The output is

$$F = \sum_k a_k(x) F_k, \quad a = K \text{softmax}(z/T).$$

The final router layer starts at zero, so all three weights start at 1.0. This makes the initial output the same as the original MKDC sum; this was checked with `torch.allclose`. This step tests adaptive weighting but does not reduce branch computation.

B. Difficulty conditioning: The decoder produces coarse predictions that are not used by the original binary loss. Their normalised predictive entropy is

$$U(p) = -\frac{p \log p + (1-p) \log(1-p)}{\log 2}.$$

The mean and maximum entropy are added to the router input together with pooled image features. Entropy is detached from back-propagation and is used only as a routing feature. It is called predictive entropy, not Bayesian epistemic uncertainty.

C. Sparse conditional execution: During inference, only the selected top-1 branch is called. Hooks on the branch convolutions verify that the other branches are skipped. During training, all branches remain available so the router can learn. The move from soft weighting to hard selection is done gradually through an annealing schedule.

D. Routing-guided pruning: The router adds parameters and control-flow cost. Therefore, routing statistics are also used to find a smaller static model. Branches with low measured value can be removed, allowing the final model to run without a router.

3.1 Experimental Design

ClinicDB [6] contains 612 images, and ColonDB [7] contains 379 images. The experiments use the 80:10:10 splits and 352×352 inputs from the checked MK-UNet polyp pipeline. In this repository version, the released

training pipeline directly supports the two polyp datasets used here. Each experiment changes one main factor at a time. Because deep supervision can change performance by itself, the entropy experiment is compared with a matched supervised adaptive control instead of the plain baseline. The final study will use five seeds per arm; the preliminary results use one seed and are treated only as early evidence.

Accuracy is measured with DICE and IoU. Reliability is measured with ECE, Brier score, and error-detection AUROC. Efficiency is measured with parameter count, FLOPs, executed branch work, and batch-1 latency. Branch-selection frequency and routing entropy are also recorded. This design helps separate representation gains, routing behavior, and real execution cost instead of relying on a single metric.

4 Expected Outcomes and Preliminary Results

The main goal is to develop a sparse MK-UNet extension that keeps segmentation quality while reducing measured inference cost. The study also aims to develop a smaller static model using routing results. The full study will test four hypotheses: (1) annealed sparse top-1 can maintain accuracy while reducing measured latency across datasets and seeds; (2) routing guided pruning can produce a smaller static model with similar accuracy after full training; (3) the benefit of adaptive routing remains after using multiple seeds and the full training budget; and (4) part of the ColonDB reproduction gap is explained by the difference between the paper’s training settings and the repository-default settings. These are testable hypotheses, not guaranteed outcomes.

4.1 Preliminary Results

All extension runs reported here used an Intel i9-14900 CPU with 32 GB RAM and no GPU. The results are preliminary and should not be treated as final multi-seed results.

The Step 1 runs used MK-UNet-T for 200 epochs with seed 42 and the default training settings from the repository. ClinicDB was close to the published value, while ColonDB was lower.

Dataset	Paper DICE	Our DICE	Difference
ClinicDB	91.26	91.09	-0.17
ColonDB	85.03	78.32	-6.71

The ClinicDB value is close to the paper’s five-run mean, but it is not an exact reproduction because the run used one seed and different training settings. The ColonDB run reached a stable plateau 6.71 DICE points below the published mean. This makes a major optimization failure less likely, but differences in the training protocol, augmentation, preprocessing, and random seed may still affect the result. A paper-exact, multi-seed ColonDB rerun is therefore an early diagnostic task.

4.2 Adaptive Routing and Difficulty Signal

The following ClinicDB runs used 100 epochs and seed 42.

Arm	Params	DICE	ECE	Error AUROC	Main factor
fixed, no aux	27,384	87.65	0.0067	0.971	baseline
fixed + aux	27,384	88.89	0.0077	0.974	deep supervision
adaptive + aux	30,588	89.97	0.0057	0.980	adaptive routing
entropy + aux	30,618	88.99	0.0067	0.972	entropy conditioning

In this single-seed setup, deep supervision increased DICE by +1.24 points. Adaptive routing then added another +1.08 points over the matched supervised control. Adding predictive entropy reduced DICE by -0.98 points compared with adaptive routing. Therefore, the entropy-conditioned router did not improve segmentation in this run. However, the entropy signal was still strongly related to error: error-detection AUROC was 0.97–0.98, ECE was below 0.008, and the per-image entropy-error correlation was $r = 0.86$. One seed is not enough to determine why a useful error signal did not improve routing.

4.3 Routing-Guided Pruning

Preliminary pruning results are shown below.

Kernels	Params	Change	DICE	DICE change
{1, 3, 5} original	27,384	–	87.65	–
{3, 5}	26,550	-3.0%	90.18	+2.53
{1, 5}	24,326	-11.2%	88.23	+0.58
{1, 3}	19,878	-27.4%	87.06	-0.59

Two pruned variants finished above the full 100-epoch model, while the most aggressive variant finished 0.59 DICE points below it. Because these runs were still improving near epoch 100, the table does not show that pruning improves accuracy. The narrower result is that parameter count can be reduced by 3–27% while the observed DICE change remains between -0.59 and +2.53 points at this reduced training budget.

4.4 Sparse Execution

The main sparse-execution measurements are shown below:

Run	DICE	FLOPs	Batch-1 latency	Depth-wise work
fixed baseline, 100 ep	87.65	0.0668 G	53.6 ms	100%
sparse top-1, abrupt, 30 ep FT	83.72	0.0307 G	–	33%
sparse top-1, annealed, 60 ep FT	90.01	0.0307 G	46.4 ms	33%
sparse top-2	–	–	59.7 ms	67%

Forward hooks showed that top-1 inference executed 10 of 30 branch convolutions for batch size 1; the other 20 were not called. The annealed 60-epoch fine-tuning run finished 6.29 DICE points above the abrupt 30-epoch run, but the schedule and fine-tuning length are both different, so this comparison does not isolate the effect of annealing. The sparse top-1 measurement reduced reported FLOPs by about 54% and batch-1 latency from 53.6 ms to 46.4 ms, a 13.3% reduction in that CPU session. Top-2 measured 59.7 ms and was slower than the dense baseline. These results show why measured latency is needed instead of using FLOPs alone.

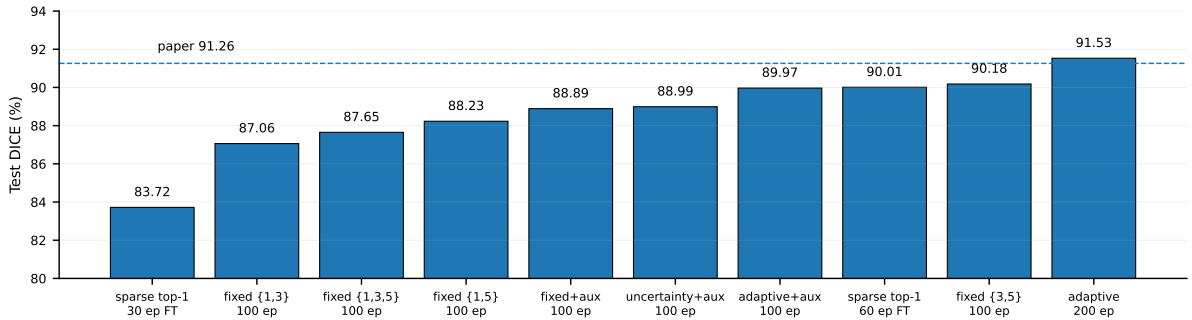


Figure 2: Preliminary ClinicDB results. Training budgets are shown under each bar. The dashed line shows the published MK-UNet-T DICE of 91.26. Runs with different training budgets are not directly comparable.

The early results help narrow the proposal, but they do not prove the final method. Soft adaptive routing is worth testing further because it gave a small gain over the matched supervised control in one run. Predictive entropy did not improve routing and should remain a secondary analysis or a negative result unless multi-seed experiments give a different result. Sparse top-1 is more directly related to the main efficiency question because it actually skips branches. Routing-guided pruning provides another option because it removes the router overhead completely.

4.5 Current Limitations and Research Plan

The current results have five main limitations: all extension results use one seed; most method comparisons use 100 epochs and are not fully converged; the extension experiments currently use only ClinicDB and MK-UNet-T; latency was measured on a desktop CPU instead of edge hardware; and the checked repository version limits direct reproduction to the polyp pipeline. These limitations are included in the next-stage plan rather than hidden.

Risk	Planned test or interpretation
Small gains are seed noise	Repeat each final arm with five seeds and report the variation instead of relying on one run.
Router selects one kernel most of the time	Use selection histograms to check for collapse; if stable, treat it as evidence for branch redundancy and test static pruning.
Annealing result does not generalise	Compare matched annealing schedules with the same fine-tuning budget across both datasets.
Pruning result reflects faster convergence	Train the pruned and full models to a converged budget before making an accuracy claim.
Sparse latency gain does not transfer	Measure the target edge hardware and report the gains as CPU-only if they do not transfer.
Entropy conditioning remains unhelpful	Report the negative result and keep the main method focused on sparse execution and pruning.

The project is planned over five months. Month 1 will update the novelty review, move the experiments to GPU, repeat the main experiments with full training budgets and five seeds, and test paper-exact versus repository-default settings on ColonDB. Month 2 will test matched annealing schedules, sparse top-1 on both datasets, edge-device latency, and lower-cost routers. Month 3 will repeat routing-guided pruning across datasets and test combined pruned and sparse models. Month 4 will study calibration, error detection, cross-dataset transfer, and qualitative routing or failure maps. Month 5 will combine the ablation results, add a second modality if a reliable data pipeline is available, and prepare the manuscript.

Even if the main accuracy hypotheses are not supported, the study can still answer whether explicit conditional branch execution is practical at the 0.03 M-parameter scale, how FLOP savings relate to real latency, and whether routing statistics can help produce a smaller static MK-UNet. Negative results, including the current entropy-routing result, will be reported rather than hidden.

References

- [1] M. M. Rahman and R. Marculescu, “MK-UNet: Multi-kernel lightweight CNN for medical image segmentation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, October 2025, pp. 1053–1062.
- [2] X. Wang, J. Yu, B. Zhang, X. Huang, X. Shen, and M. Xia, “LightAWNNet: Lightweight adaptive weighting network based on dynamic convolutions for medical image segmentation,” *Journal of Applied Clinical Medical Physics*, vol. 26, no. 2, p. e14584, 2025.
- [3] Y. Wang, H. Wang, and F. Zhang, “A medical image segmentation model with auto-dynamic convolution and location attention mechanism,” *Computer Methods and Programs in Biomedicine*, vol. 261, p. 108593, 2025.
- [4] Q. Hao, L. Yu, S. Tian, C. Zhou, Y. Yu, and L. Zhang, “SEDyConv: Spatially enhanced multi-dimensional dynamic convolution for medical multi-organ segmentation in CTs,” *Knowledge-Based Systems*, vol. 323, p. 113707, 2025.
- [5] B. Shi, W.-p. Zhu, and M. Swamy, “SGDC: Structurally-guided dynamic convolution for medical image segmentation,” in *Proceedings of the 9th International Conference on Medical Imaging with Deep Learning*, ser. Proceedings of Machine Learning Research, vol. 315. PMLR, 2026, pp. 989–1003.
- [6] J. Bernal, F. J. Sánchez, G. Fernández-Esparrach, D. Gil, C. Rodríguez, and F. Vilarino, “WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation versus saliency maps from physicians,” *Computerized Medical Imaging and Graphics*, vol. 43, pp. 99–111, 2015.
- [7] D. Vázquez, J. Bernal, F. J. Sánchez, G. Fernández-Esparrach, A. M. López, A. Romero, M. Drozdal, and A. Courville, “A benchmark for endoluminal scene segmentation of colonoscopy images,” *Journal of Healthcare Engineering*, vol. 2017, p. 4037190, 2017.